

# 두리

## AI 기반 금융 혜택 통합 전자지갑

카드 실적과 포인트, 멤버십을 한 곳에 모으고 결제 직전에 가장 유리한 카드를 골라주는 모바일 전자지갑입니다.

문제를 정의하고, 화면을 설계하고, 코드로 구현하고, 결과를 검증하기까지 한 사람이 이어서 맡았습니다.



## 장은영

서비스 기획 · UX/UI 설계 · 프론트엔드 개발

### PROJECT

KB국민은행 주관 부트캠프 파이널

### PERIOD

2026.07.09 - 08.26

### TEAM

4명 · 팀장 / PM

### ROLE

기획 · 설계 · 프론트엔드 전담

## 프로젝트 개요와 담당 범위

4명이 세 개 서버를 나눠 만들었고, 저는 기획부터 프론트엔드 구현까지 제품의 사용자 접점 전체를 맡았습니다.

22

화면

61

컴포넌트

137

디자인 토큰

28

단위 테스트

### ■ 담당 범위

|      |                                                                    |
|------|--------------------------------------------------------------------|
| 기획   | 문제 정의, 사용자 설문(N=34) 설계·집행·분석, 페르소나, 경쟁 서비스 분석, 요구사항 정의, 일정 관리(WBS) |
| 설계   | 정보구조·화면 설계 22종, 디자인 시스템(토큰 137개) 구축, 라이트/다크 테마 설계, 접근성 기준 수립·실측    |
| 개발   | 프론트엔드 전담 — Vue 3 SPA 전체 구현, 상태 관리, API 연동, 단위 테스트 작성               |
| 팀 리드 | 팀장·PM·발표. ERD·API 명세·엔티티 보고서·테이블 정의서 중간/최종 검토, 협업 규약 수립            |

### ■ 기술 스택

|        |              |            |       |       |        |       |
|--------|--------------|------------|-------|-------|--------|-------|
| Vue 3  | Vite         | Vue Router | Pinia | Axios | Vitest | Figma |
| Notion | Git · GitHub |            |       |       |        |       |

UI 프레임워크는 쓰지 않았습니다. Tailwind·Bootstrap·Vuetify 없이 CSS를 직접 작성하고, 색·간격·타이포는 전부 토큰 파일의 CSS 변수를 참조합니다.

### ■ 시스템 구성

|           |                                           |
|-----------|-------------------------------------------|
| Frontend  | Vue.js SPA ← 담당                           |
| Backend   | Spring Legacy · MyBatis · MySQL 8 · Redis |
| AI Server | FastAPI · OpenAI API                      |

### ■ 이 프로젝트에서 내린 판단

| 상황        | 선택                  | 이유                                |
|-----------|---------------------|-----------------------------------|
| UI 프레임워크  | 사용하지 않음             | 토큰 기반 디자인 시스템을 직접 통제하려고           |
| 결제 카드 캐러셀 | 라이브러리 없이 직접 구현      | 제스처 분기와 2단계 연출을 범용 캐러셀로 제어할 수 없어서 |
| 브랜드 컬러 반전 | 값만 교체, 토큰명 유지       | 전면 리네이밍의 회귀 위험이 남은 기간 대비 크다고 판단   |
| 인증 체크 위치  | 화면별 검사 대신 전역 라우터 가드 | 화면이 늘어도 규칙이 한 곳에만 있도록             |
| 명암비       | 토큰 확정 단계에서 실측       | 구현을 마친 뒤 발견하면 팔레트를 다시 뒤집어야 해서     |
| 테스트 범위    | 5개 파일 28개           | 커버리지가 아니라 "틀리면 다른 곳까지 들어지는 자리" 기준 |

### ■ 이 문서의 구성

기획 문제 정의와 근거(3-5p) → 설계 디자인 시스템과 접근성(6-7p) → 구현 인터랙션·구조·품질(8-9p) → 검증 트러블슈팅과 협업(10-11p)

## 문제 정의와 사용자 조사

정보가 없어서가 아니라, 필요한 순간에 없어서였습니다.

"카드나 멤버십마다 앱이 따로 있어서 혜택을 확인하려면 여러 앱을 번갈아 들어가야 하는 점이 가장 불편했습니다." — 설문 주관식 응답 中

### ■ 설문 설계

「카드 혜택 관리 경험 조사」 · 구글폼 · 2026.08.11~08.15 · 응답 34건 · 객관식 5문항 + 주관식 2문항. 기획 방향을 정하기 전에 가설을 검증할 목적으로 직접 설계하고 돌렸습니다.

**97%**

카드 2장 이상 보유

**88%**

혜택 관리가 복잡하다

**91%**

혜택·포인트를 놓친 경험

**91%**

통합 관리 앱이 필요하다

### ■ 주관식 41건을 기능으로 연결

응답을 네 갈래로 묶고 각 갈래를 기능 하나에 대응시켰습니다. 스코프 밖 응답은 제외했다는 것까지 남겼습니다.

| 응답 갈래                  | 비중       | 연결한 기능              |
|------------------------|----------|---------------------|
| 정보가 많아 판단이 안 된다        | 9건 · 43% | 가맹점별 카드·멤버십 추천      |
| 소멸을 알려주지 않는다           | 3건 · 14% | 내 이용 이력 기반 요약·알림    |
| 쓸 카드를 일일이 찾아야 한다       | 3건 · 14% | 카드·멤버십 혜택 통합 비교     |
| 앱을 여러 개 오가야 한다         | 2건 · 10% | 각 멤버십 관리 페이지로 바로 이동 |
| 범위 밖 (지역 제한, 개인화 부족 등) | 4건 · 19% | 이번 스코프에서 제외         |

### ■ 페르소나 — 연령이 아니라 상황으로 정의

#### 시현 · 직장인

카드 4~5장 + 멤버십 1~2개 보유. 혜택은 알지만 챙기지는 못하는 사용자.

#### 71%가 해당

응답자 34명 중 24명이 카드 2장 이상 & 멤버십 1개 이상.

#### 카드 장수는 변수가 아니었다

혜택을 놓친 경험 비율이 1~3장 93%, 4장 이상 89%로 거의 같음. 그래서 연령·보유량 제한을 두지 않음.

### ■ 기대효과 — 설문에서 나온 문제와 1:1로 대응

#### 앱 여러 개 → 1개

카드사·멤버십 앱을 오갈 필요가 없습니다.

#### 결제 이후 → 결제 직전

실적 확인이 아니라 카드 선택 시점에 개입합니다.

#### 놓친 혜택 → 회수한 혜택

쓰지 못하던 혜택을 미리 알려줍니다.

## 경쟁 분석과 차별점

새 카드를 발급받기 전이 아니라, 이미 가진 카드를 쓰는 순간에 개입하기로 했습니다.

### ■ 유사 서비스 비교

| 질문           | 기존 서비스                          | 두리의 선택                  |
|--------------|---------------------------------|-------------------------|
| 언제 추천하나      | 토스 — 결제 이후 실적을 확인해주는 방식         | 결제 전, 어떤 카드를 쓸지 먼저 추천   |
| 어디까지 다루나     | KB Pay · 카카오페이 — 자사 카드 생태계 안에서만 | 카드사 상관없이 보유한 모든 카드·멤버십  |
| 어떻게 추천하나     | 삼성월렛 — 마지막에 쓴 카드를 먼저 노출         | 결제마다 유리한 카드를 근거와 함께     |
| 무엇을 위한 서비스인가 | 네이버페이 ·뱅크샐러드 — 새 카드 탐색·비교       | 이미 보유한 카드를 최적으로 쓰는 데 집중 |

### ■ 핵심 가치 세 가지

#### 한 곳에서

보유 카드·멤버십을 한 번 등록해두면 목록으로 관리됩니다.

#### 두리번 없이

이 가맹점에서 어떤 멤버십을 적립하면 유리한지 바로 알려줍니다.

#### 물어보면

약관을 읽지 않아도 대화로 내 혜택을 확인할 수 있습니다.

### ■ 기능 우선순위 — 지갑을 먼저 보고, 물어보면 밝을 본다

#### STEP 01 · 핵심 기능

먼저 지갑을 봅니다. 결제 직전, 이미 가진 카드와 멤버십 중 지금 이 가맹점에서 가장 유리한 것을 실적·한도까지 계산해 근거와 함께 알려줍니다.

#### STEP 02 · 보조 기능

물어보면 밝도 봅니다. 챗봇에 물으면 보유하지 않은 카드의 혜택도 안내합니다. 다만 먼저 권한지는 않습니다.

### ■ 기획 단계에서 걸러낸 것

- 초기 아이디어였던 **가족 혜택 대시보드**는 "타인의 카드 사용을 유도하는 기능"이 여신전문금융업법에 저촉될 수 있어, 결제 유도가 아닌 **정보 공유형**으로 방향을 조정했습니다.
- 추천 결과 문구를 "**AI 추천**"이 아니라 "**추천 결과 / 추천 근거**"로 표기하도록 정했습니다. 숫자를 계산하는 것은 추천 엔진이고 LLM은 결과를 문장으로 바꾸는 역할만 하기 때문입니다.

## 사용자 여정과 예외 설계

화면을 예쁘게 만드는 것과, 사용자가 막히지 않게 만드는 것은 다른 일이라고 봤습니다.

### ■ 등록 상태에 따라 열리는 기능이 달라집니다

| 단계     | 상태     | 화면이 하는 일                             |
|--------|--------|--------------------------------------|
| STEP 1 | 비로그인   | 서비스 소개만 노출. 기능 대신 무엇을 하는 앱을지를 먼저 보여줌 |
| STEP 2 | 로그인    | 가입은 했지만 아직 아무것도 등록하지 않은 상태 — 등록으로 유도 |
| STEP 3 | 카드만 등록 | 카드 기능 활성화, 멤버십은 등록을 권유               |
| STEP 4 | 완전 등록  | 대시보드 풀 기능 노출                         |

### ■ 막히는 지점을 먼저 그렸습니다

| 상황        | 벌어지는 일                  | 내린 판단                                 |
|-----------|-------------------------|---------------------------------------|
| 비어 있을 때   | 카드는 등록했는데 혜택 데이터가 아직 없음 | 화면 전체를 비우지 않고 컴포넌트마다 따로 분기 — 있는 건 보이게 |
| 틀릴 수 있을 때 | 결제 금액을 모르는 채로 결제하려 함    | 금액을 선택 입력으로 — 모르면 모르는 대로 진행 가능        |
| 너무 많을 때   | 고정 카드를 계속 늘리면 홈이 무의미해짐  | 3개 제한 + 초과 시 토스트로 이유를 설명              |
| 처음일 때     | 스вай프가 되는지 아무도 모름       | 최초 방문 1회만 안내 토스트 — 매번은 아님             |

### ■ 주요 화면 7종

|                     |                                 |                  |
|---------------------|---------------------------------|------------------|
| 홈 · 실적·혜택 요약 브리핑    | 카드 · 등록·관리·실적 현황                | 결제 · 추천 + QR 결제  |
| 소비내역 · 기간·조건 필터 조회  | 혜택 · 리포트·포인트·멤버십                | AI 챗봇 · 혜택·약관 질의 |
| 알림 · 실적·혜택 상태 자동 안내 | 설정·개인화·회원탈퇴 등 부속 화면 포함 총 22개 화면 |                  |

### ■ 7주 일정

| W1                                     | W2-W3                                               | W4-W5                                    | W6                                       | W7                               |
|----------------------------------------|-----------------------------------------------------|------------------------------------------|------------------------------------------|----------------------------------|
| <b>기획</b><br>주제 선정<br>시장 조사<br>요구사항 정의 | <b>설계</b><br>화면 설계<br>디자인 시스템<br>ERD · API 명세 · WBS | <b>개발</b><br>회원/인증<br>카드 · 결제<br>혜택 · 챗봇 | <b>통합</b><br>화면 단위 통합<br>API 연동<br>버그 수정 | <b>마무리</b><br>산출물 정리<br>시연·발표 준비 |

## 디자인 토큰과 테마

색을 정하는 사람과 구현하는 사람이 같았기 때문에, 디자인 결정을 코드 한 파일에서 끝낼 수 있었습니다.

### ■ 토큰 137개

| 토큰 그룹               | 개수         |
|---------------------|------------|
| Color               | 51         |
| Typography          | 48         |
| Font weight / size  | 11         |
| Spacing             | 7          |
| Radius · Z-index    | 12         |
| Shadow · Transition | 8          |
| <b>:root 합계</b>     | <b>137</b> |
| 다크모드 재정의            | 31         |

다크모드는 재정의가 필요한 31개만 덮어씁니다. 나머지 106개는 두 모드가 공유합니다.

### ■ 색마다 역할을 고정

| 역할                 | Light   | Dark    |
|--------------------|---------|---------|
| Primary 골드 — 버튼·강조 | #EFDD39 | #FFEB3B |
| Secondary 그린 — 멤버십 | #3D6B52 | #4B8B70 |
| Nav Active         | #7AB88A | #4B8B70 |
| Error 코랄 — 경고      | #A84E68 | #D17B93 |
| Text Primary       | #24242A | #EDEFF1 |
| Background         | #F7F8FA | #1A1A1C |

규칙 — 골드는 포인트, 그린은 멤버십. 두리가 다루는 두 축을 색으로 나눴습니다.

### ■ 브랜드 컬러를 뒤집고, 그 결정이 만든 문제를 처리한 과정

#### ① 결정

그린 → 골드. 차분함을 유지하면서 놀려야 할 곳만 분명히 드러나는 색으로 판단. 그린은 버리지 않고 헤택·멤버십 강조 역할 변경.

#### ② 발견

바탕이 밝아지며 골드 위 흰 글씨가 **1.39:1**. 눈으로는 괜찮아 보였지만 재보니 기준의 1/3이었습니다.

#### ③ 조정

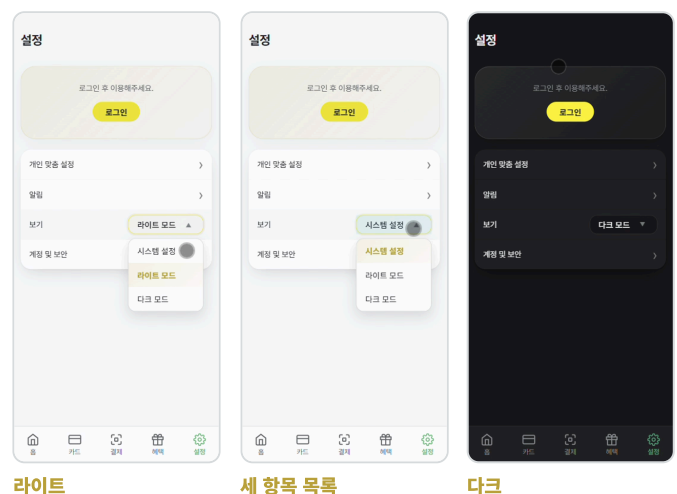
버튼 텍스트를 #24242A로 교체해 **11.08:1** 확보. 같은 기준으로 나머지 색도 전부 다시 측정.

토큰 --color-gold-\* 이름은 참조가 많아 유지하고 값만 교체했습니다. 전면 리네이밍의 회귀 위험이 더 크다고 판단해 **의도한 부채임을 주석**으로 남겼습니다.

### ■ 테마 전환 — 저장하는 값과 적용하는 값을 구분

고르는 값은 system / light / dark 세 가지, 실제 적용값은 light / dark 두 가지입니다. 이 둘을 같은 값으로 다루면 버그가 납니다. 설정 UI가 토글이 아니라 세 항목 목록인 이유이기도 합니다.

```
// 우선순위
1 data-theme          사용자가 고른 값
2 prefers-color-scheme OS 설정
3 :root               기본 라이트
```



라이트

세 항목 목록

다크

## 접근성 — 감이 아니라 수치로

본문 4.5:1, UI 요소 3:1을 기준으로 각 요소를 실제 배경 토큰과 짝지어 측정했습니다. 미달 항목도 그대로 남겼습니다.

### ■ 명암비 실측

| 조합             | 대비     | 판정  |
|----------------|--------|-----|
| 본문 (라이트)       | 14.5:1 | AAA |
| 본문 (다크)        | 15.1:1 | AAA |
| 버튼 텍스트         | 11.1:1 | AAA |
| 보조 (라이트)       | 5.4:1  | AA  |
| 보조 (다크)        | 5.2:1  | AA  |
| Coral 텍스트      | 5.3:1  | AA  |
| Tertiary (라이트) | 3.7:1  | 미달  |
| 하단 탭 활성화       | 2.3:1  | 미달  |

### ■ 터치 영역 — 최소 44×44

| 요소            | 크기        | 판정 |
|---------------|-----------|----|
| 기본 버튼 (large) | 100% × 52 | 충족 |
| 입력 필드         | 100% × 48 | 충족 |
| 하단 탭 1칸       | 96 × 70   | 충족 |
| 토스트 닫기        | 24 → 44   | 확장 |
| 작은 버튼 (small) | auto × 36 | 미달 |
| 모달 닫기 X       | 28 × 28   | 미달 |

토스트 닫기는 아이콘을 24px로 유지한 채 ::after로 히트박스만 44px까지 넓혔습니다. 아이콘을 키우면 레이아웃이 흔들리기 때문입니다.

### ■ 색에만 의존하지 않는 신호

| 상태      | 색 외의 신호                    | 상태     | 색 외의 신호                    |
|---------|----------------------------|--------|----------------------------|
| 입력 오류   | 테두리 + 안내 문구 + aria-invalid | 토스트    | role="alert" + aria-live   |
| 키보드 포커스 | 테두리 + 3px 포커스 링            | 모달     | role="dialog" + aria-modal |
| 적립 성공   | 색 + '적립 완료' 라벨             | 아이콘 버튼 | aria-label                 |
| 추천 1위   | 색 + 순위 숫자 병기               |        |                            |

### ■ 토스트는 종류에 따라 알림 강도를 나눴습니다

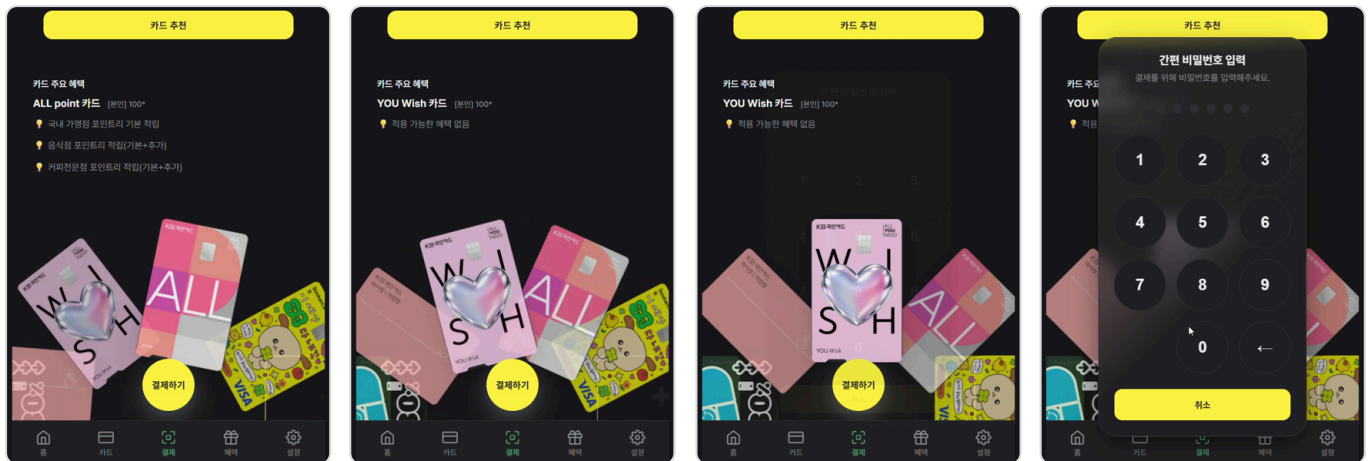
```
role="alert"
:aria-live="type === 'error' ? 'assertive' : 'polite'"
// 오류는 읽던 내용을 끊고서라도 알려야 하고, 일반 알림은 끊지 않아야 한다.
// 둘 다 assertive면 화면 낭독기 사용자에게 사소한 알림이 계속 끼어든다.
```

### ■ 아직 못 고친 것

전수 점검은 했지만 마감까지 다 고치지 못했습니다. Tertiary 텍스트와 하단 탭 활성화색은 브랜드 컬러와 얹혀 있어 토큰 재조정이 필요한 범위였고, small 버튼과 모달 닫기는 토스트에 적용한 히트박스 방식을 그대로 쓰면 됩니다. 측정해서 목록으로 남겨두면 다음에 손댈 때 다시 재지 않아도 됩니다.

## 결제 카드 캐러셀

"카드를 부채꼴로 펼쳐 고르고, 위로 밀어 올려 결제한다"를 라이브러리 없이 직접 구현했습니다.



**01** 카드 개수에 맞춰 각도·간격 자동 분배. **02** 좌우 스와이프로 카드 전환 최상단만 선명, 나머지는 반투명

**03** 위로 밀면 카드가 올라오며 뒤집힘

**04** 진행도 50% 넘기면 비밀번호 모달로 연결

### ■ 라이브러리로 풀리지 않았던 요구

| 요구                                           | 해결                                                                      |
|----------------------------------------------|-------------------------------------------------------------------------|
| 카드 개수(1 / 2 / 3장 이상)마다 부채꼴 각도와 간격이 달라져야 함    | 개수별 레이아웃 분기. 임계값·타이밍·각도·간격 <b>27개를 상수 객체 하나로</b> 분리해 로직 수정 없이 숫자만 바꿔 조정 |
| 위로 밀기(플립)와 좌우 밀기(회전)를 한 제스처 스트림에서 구분         | 세로/가로 이동으로 분기. 손가락 입력이 불규칙해 카드가 떨어져 <b>현재값 30% + 직전값 70%</b> 로 보정       |
| 추천 결과가 오면 카드가 중앙으로 모였다가 추천순으로 다시 펼쳐지는 2단계 연결 | easeInOutCubic 이징을 직접 구현하고 requestAnimationFrame으로 800ms 수렴 → 재정렬 → 재전개 |

### ■ 되돌아가는 길을 하나로

결제가 성공하면 취소되는 타임아웃이 나온 카드는 원래 자리로 돌아와야 합니다. 분기마다 역방향 애니메이션을 만들면 경우의 수가 늘수록 빠지는 조합이 생깁니다. 초기화 함수 하나를 네 경로가 모두 호출하고, 복귀 동작은 CSS transition에 맡겼습니다.

```
// 성공 · 닫기 · 취소 · 타임아웃 - 네 경로 모두
resetInteractionState()
flipState = { progress: 0, index: null, ... }
cardUpOffset = 0
```

### ■ 손대지 않아도 되게

카드 도안은 가로형과 세로형이 섞여 들어옵니다. 카드마다 방향을 코드에 적어두면 카드가 추가될 때마다 코드를 고쳐야 하므로, **로드된 이미지의 실제 크기로 판정**하게 했습니다. 카드가 늘어도 손댈 곳이 없습니다.



**05** 간편 비밀번호

**06** QR 인증 · 60초

**07** 성공 후 자동 복귀



## 구조와 품질

서버와 만나는 지점을 좁히고, 틀리기 쉬운 자리에만 테스트를 붙였습니다.

### ■ 서버와 만나는 지점을 함수 하나로

| 지점                             | 규칙                                                                      |
|--------------------------------|-------------------------------------------------------------------------|
| <code>unwrapResponse()</code>  | 성공/실패 봉투를 푸는 <b>유일한</b> 자리. 화면은 문구가 아니라 code로 분기하므로 에러 객체에 code를 반드시 남김 |
| <code>getErrorMessage()</code> | 서버 메시지 → 예외 메시지 → 화면 기본 문구 순으로 사용자에게 보일 오류 문구를 결정                       |
| axios 인터셉터                     | 토큰 자동 첨부. 401이면 재발급 후 원래 요청 재시도                                         |
| <code>router.beforeEach</code> | 보호 라우트 17개를 <code>meta.requiresAuth</code> 로 선언하고 한 곳에서 판단              |

### ■ 토큰이 만료돼도 요청을 잃지 않게

여러 요청이 동시에 401을 맞으면 재발급이 중복 호출됩니다. 재발급 중에 들어온 요청은 큐에 넣었다가 새 토큰으로 한 번에 재실행합니다.

```
if (isRefreshing) {
  return new Promise((resolve) => {
    addRefreshQueue((newToken) => {
      // 새 토큰으로 원래 요청을 다시 보낸다
      resolve(api(originalRequest))
    })
  })
}
```

### ■ 일부가 실패해도 화면은 살립니다

카드 목록 화면은 기본 목록 API와 월별 실적 API 두 개를 합쳐 그림니다. 실적 조회가 실패했다고 카드 목록까지 빈 화면이 되면 안 된다고 봤습니다.

```
cards = basicCards // 먼저 채우고
try {
  cards = merge(실적) // 오면 덮어쓴다
} catch {
  // 실패해도 기본 목록은 유지
}
```

### ■ 테스트는 커버리지가 아니라 "틀리면 다른 곳까지 들어지는 자리" 기준

| 대상                                             | 개수        | 고른 이유                         |
|------------------------------------------------|-----------|-------------------------------|
| <code>unwrapResponse</code>                    | 4         | 모든 API 호출이 통과하는 유일한 지점        |
| <code>getErrorMessage</code>                   | 4         | 사용자가 보는 오류 문구를 결정             |
| <code>validatePassword · maskCardNumber</code> | 7         | 규칙이 명확해 회귀가 즉시 드러남            |
| <code>resolveTheme · saveTheme</code>          | 6         | <b>실제로 한 번 틀렸던 곳</b> — 회귀 테스트 |
| <code>PaymentPasswordModal</code>              | 7         | 결제 관문. 서버 응답별 화면 반응 검증        |
| <b>Vitest + jsdom · 5개 파일</b>                  | <b>28</b> | 환경 구성부터 직접 작성                 |

테스트는 내부 상태를 건드리지 않고 **사용자가 하는 동작 그대로** 작성했습니다. 숫자패드 버튼을 여섯 번 클릭해 비밀번호를 채우는 식입니다. 내부 상태를 직접 바꾸면 "버튼이 함수에 연결되지 않은" 종류의 버그를 놓칩니다.

## 트러블슈팅

대부분은 기능을 만들다 막힌 게 아니라, 만들어 둔 것을 문서와 대조하며 다시 점검하다 나왔습니다.

| 문제                        | 원인                                                                          | 해결                                                                                      |
|---------------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| '시스템 설정'을 골라도 항상 라이트로 적용  | OS를 확인하지 않고 무조건 라이트로 처리. <b>팀원 전원의 OS가 라이트</b> 여서 결과가 우연히 맞아떨어져 아무도 발견하지 못함 | matchMedia로 OS 값을 읽어 환산. 같은 실수가 다시 들어오지 않도록 <b>회귀 테스트 6개</b> 추가                         |
| 라우터에는 있는데 들어갈 수 없는 화면 10개 | 페이지를 바텀시트로 전환하면서 옛 화면 파일을 지우지 않음                                            | 파일 이름이 아니라 <b>import 기준으로 대조</b> 해 확정 후 삭제. 이름이 비슷한 화면이 많아 이름으로 지웠다면 살아 있는 화면을 지울 뻔했습니다 |
| 로그인하지 않아도 URL만 알면 진입 가능   | 공통 가드 없이 화면마다 개별로 확인<br>→ 확인이 빠진 화면이 생김                                     | 라우트에 <code>meta.requiresAuth</code> 를 선언하고 전역 가드 한 곳으로 통합 (보호 17 / 공개 5)                |
| 같은 추천 기능인데 화면마다 반환 형태가 다름 | 화면을 만들 때마다 API 호출 함수를 각자 추가                                                 | 공용 함수로 통합. 지금 <code>unwrapResponse</code> 를 유일한 통로로 두는 규칙은 이 일을 겪고 정한 것입니다              |
| 결제 금액을 비우면 QR 생성 실패       | 기획을 "금액은 선택 입력"으로 바꿨는데 DB 컬럼은 필수값으로 남아 있었음                                  | 백엔드와 함께 서버 검증 완화 + 저장 전 <code>null</code> → <code>0</code> 변환                           |

### ■ 어떻게 찾았나 — 문서와 코드를 대조하는 점검

UX/UI 설명서를 다 쓴 뒤, **문서에 적은 내용이 실제 코드와 맞는지 전수 대조**하는 점검을 돌렸습니다. 위 다섯 건 중 셋이 그 과정에서 나왔습니다. 기능을 만들 때는 "동작한다"에서 멈추게 되는데, 문서에 적어둔 명세와 나란히 놓으면 **동작하지만 명세와 다른 것**이 드러납니다. 테마 버그와 인증 구멍이 그렇게 잡혔습니다.

#### 여기서 배운 것 ①

개발 환경이 한쪽으로 쏠려 있으면 버그가 정답처럼 보입니다. 팀원 넷의 OS가 모두 라이트였기 때문에 테마 버그는 몇 주 동안 "정상"이었습니다. 그 뒤로는 테마 작업을 할 때 OS 설정을 바꿔가며 확인합니다.

#### 여기서 배운 것 ②

기획 변경은 화면에서 끝나지 않고 스키마까지 내려갑니다. "금액을 선택 입력으로" 한 줄이 DB 제약과 서버 검증까지 바뀌어야 하는 일이었습니다. 이후로는 기획을 바꿀 때 영향 범위를 레이어 별로 적어두고 공유합니다.

## 팀을 굴러가게 만든 규칙

4명이 세 개 서버를 나눠 만들었기 때문에, 코드가 섞이는 지점을 규칙으로 먼저 막았습니다.

### ■ 설계 산출물 검토

스키마를 직접 작성하지는 않았습니다. 대신 설계 산출물 네 가지가 전부 제 검토를 거쳤습니다. 한 번에 몰아서 보지 않고 **중간 단계에서 한 번, 최종본에서 한 번** 봤습니다.

| 산출물               | 관여                                     |
|-------------------|----------------------------------------|
| ERD               | 중간·최종 검토, PK/FK 정합성 문제는 수정 요청 또는 직접 수정 |
| API 명세            | 중간·최종 검토, 일부 직접 수정                     |
| 엔티티 보고서 · 테이블 정의서 | 중간·최종 검토                               |

프론트엔드에서 데이터를 붙이는 쪽이 명세와 실제 응답의 어긋남을 가장 먼저 만납니다. 이 검토는 팀장 일이면서 동시에 제 구현을 위한 일이었습니다.

### ■ 브랜치와 리뷰

|        |                                                     |
|--------|-----------------------------------------------------|
| 브랜치    | main 직접 push 금지, develop에서만 merge, 작업은 feature/fix/ |
| 머지     | 승인 1명 + CI 통과, squash                               |
| PR 템플릿 | 무엇을 · 왜 · 변경 사항 · <b>다른 파트 영향</b> · 리뷰어가 볼 곳        |
| 커밋     | feat · fix · docs · chore · refactor · perf         |
| CI     | 세 파트 병렬 — 프론트 빌드 / 실제 MySQL 8 / DB 연결 pytest        |

PR 템플릿의 "**다른 파트 영향**" 칸이 실제로 효과가 있었습니다. 서버가 셋으로 나뉘어 한쪽 변경이 다른 쪽 연동을 깨는 일이 잦았는데, 이 칸을 채우면서 미리 걸러졌습니다.

### ■ API 규약을 문서로 고정

|       |                                      |
|-------|--------------------------------------|
| 응답 봉투 | code로 분기 — message 문자열 분기 금지         |
| 인증    | 모든 요청 Bearer JWT, 회원 ID는 토큰에서 서버가 추출 |
| 필드    | API JSON은 camelCase                  |
| 금액    | 원 단위 정수, 소수점 없음                      |
| 소유권   | 타인 리소스 요청은 403이 아닌 404               |
| 표기    | "AI 추천"이 아니라 "추천 결과 / 추천 근거"         |

### ■ 통합 전에 확인한 것

- 시연 전 검증 절차를 돌려, 자연어 질의 기능이 질문과 맞지 않는 답을 반환하는 것을 발견하고 담당자·강사에게 공유했습니다.
- 기획 단계에서 법적 리스크를 먼저 확인해 기능 방향을 조정했습니다(여신전문금융업법).
- UX/UI 설명서와 발표 자료를 앱의 **실제 디자인 토큰 값만** 써서 제작해, 문서와 제품이 하나의 시스템으로 읽히도록 했습니다.

## 회고와 다음

### 가장 많이 배운 것은 결정을 남기는 법이었습니다

토큰 이름을 그대로 둔 이유, 리다이렉트를 1.5초 늦춘 이유, 실적 API 실패를 삼키기로 한 이유는 코드만 봐서는 알 수 없습니다. 남겨두지 않으면 다음 사람이 되돌립니다. 그래서 이 프로젝트의 코드에는 무엇을 하는지보다 왜 그렇게 했는지가 더 많이 적혀 있습니다.

### 남은 과제 — 알고 남긴 것들

| 항목           | 현재                        | 다음 단계                          |
|--------------|---------------------------|--------------------------------|
| API base URL | localhost:8080 하드코딩       | 환경변수 기반 설정으로 분리                |
| 라우팅          | 인증 가드는 적용, 전 화면 즉시 import | 라우트 단위 lazy loading으로 초기 로딩 분할 |
| 접근성          | 미달 4건 파악 완료               | 대비 조정, 터치 영역 44px 확장           |
| 테스트          | 핵심 5개 파일 28개              | 결제 플로우 통합 테스트로 확장              |
| 린트           | 설정 없음                     | ESLint + Prettier 도입           |

### 다른 프로젝트

#### 룰루 (Rulelu) · 기획 · 진행 중

"룰루, 우리 사이엔 룰이 있어요" — AI가 사람을 쥐락펴락하지 않고 결정권을 사용자에게 돌려주는 대화 서비스. 아침하지도, 방 어적으로 회피하지도 않는 중간 지점을 노립니다.

기획·설계 초안 v0.1 정리 완료 — 대화 룰 6개, 의도 판별 4유형 (화풀이·판단 요청·확인 요청·무기 확보)과 판별 신호, B2B 그룹웨어 설계, 프라이버시·집계 익명성 원칙. 현재 판별 로직 프로토타입(API 연동) 준비 중.

#### 피또 (Fitto) · 1인 개발

개인 건강 관리 앱. 기획·설계·구현을 혼자 진행했습니다. 온보딩, 홈 대시보드, 식단·건강·설정 탭과 규칙 기반 로직까지 구현.

Next.js 15 App Router · Tailwind v4 · Recharts · Framer Motion. 물방울 마스코트는 AI 생성본을 버리고 Figma에서 직접 일러스트로 다시 그려, 선 기반 플랫 스타일에 성격 3종(친근·엄격·차분) 변형으로 설계했습니다.

#### CONTACT

이름                      이메일                      GitHub  
 장은영                      jey4258@gmail.com                      github.com/ <링크>